Data → Processing → Information
(Raw facts).

**Data:-** It is defined as collection of individual facts or statistics.

* It can be of many types
→ text, images, numbers, graphs, symbols, observations.
* Data can be simple & it is useless until it is analyzed, organized & interpreted.

**Types**

1) Quantitative (numerical form of data).
2) Qualitative (non-numerical).

**Information:-**

It is defined as knowledge gained through study, communication, research, or instruction.
* It is the result of analyzing & interpreting pieces of data.

| Data | Information |
|---|---|
| * collection of facts | * Puts those facts into context. |
| * Raw & unorganized | * organized. |
| * Does not depend on information. | * depends on data. |
| * Not sufficient for decision making | * we make decisions based on information. |

* The First database is known as the Integrated Data Store, or IDS.

Two models of database

→ hierarchial model (data is organised like a family tree).

→ network model (a record to have more than one parent & child record).

* A relational database is one that shows the relationship between different data records.

Types of DBMS

1.) Hierarchial (Tree like structure)
2.) Network (Graph structure, can hold multiple parent & child records).
3.) In-memory (Data is stored directly in RAM).
4.) Time-Series ~~Databas~~ (Optimal for data that changes over time).
5.) Object-oriented (data stored as objects & attributes)
6.) No SQL (Not only structured Query Language). (Documents)
7.) Relational (Tables (rows & columns), data is stored in relations).
8.) New SQL (modern relational database system).
9.) Cloud Database (Hosted by cloud, managed by cloud provider like (AWS, Azure, GCP)).
10.) Distribute Database.

(Data is stored across multiple physical locations, i.e; across different servers, regions or even continents).

## Two approaches for storing data in computers

→ File based approach
→ Database - approach.

* A file system is used for storing & organizing computer files & the data they contain to make it easy to find & access them.

**Database :-** It is a collection of related data, which contains information about one particular organization.

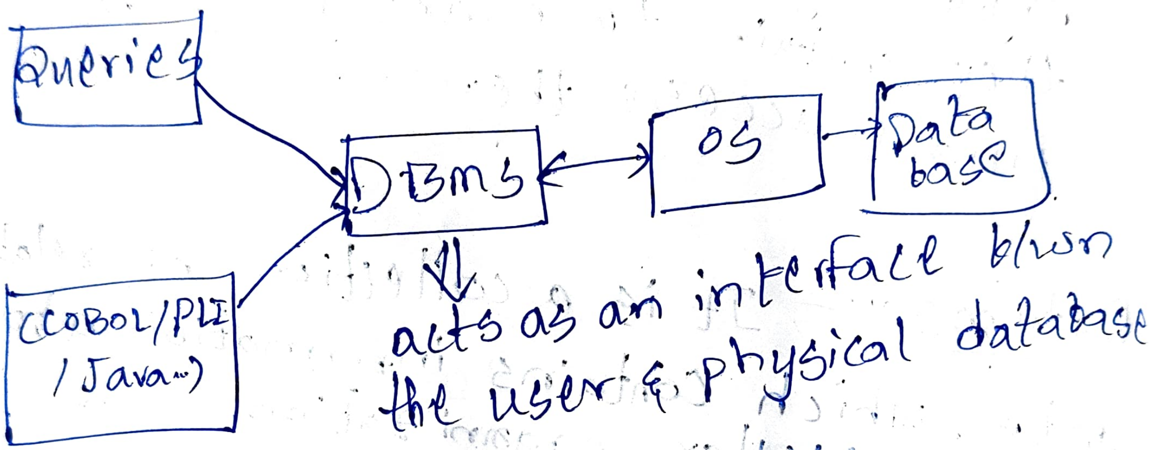
* DBMS is a software solution designed to efficiently manage, organize & retrieve data in a structured manner
* DBMS plays a vital role in supporting data driven decision-making.

### Operations on Databases

→ Adds new information
→ To view or retrieve the stored information
→ To modify or edit the existing.
→ To delete the unwanted information.
→ Arranging the information in a desired order.

# Components of Database

1) Data
2) Hardware
3) Software
4) Users.

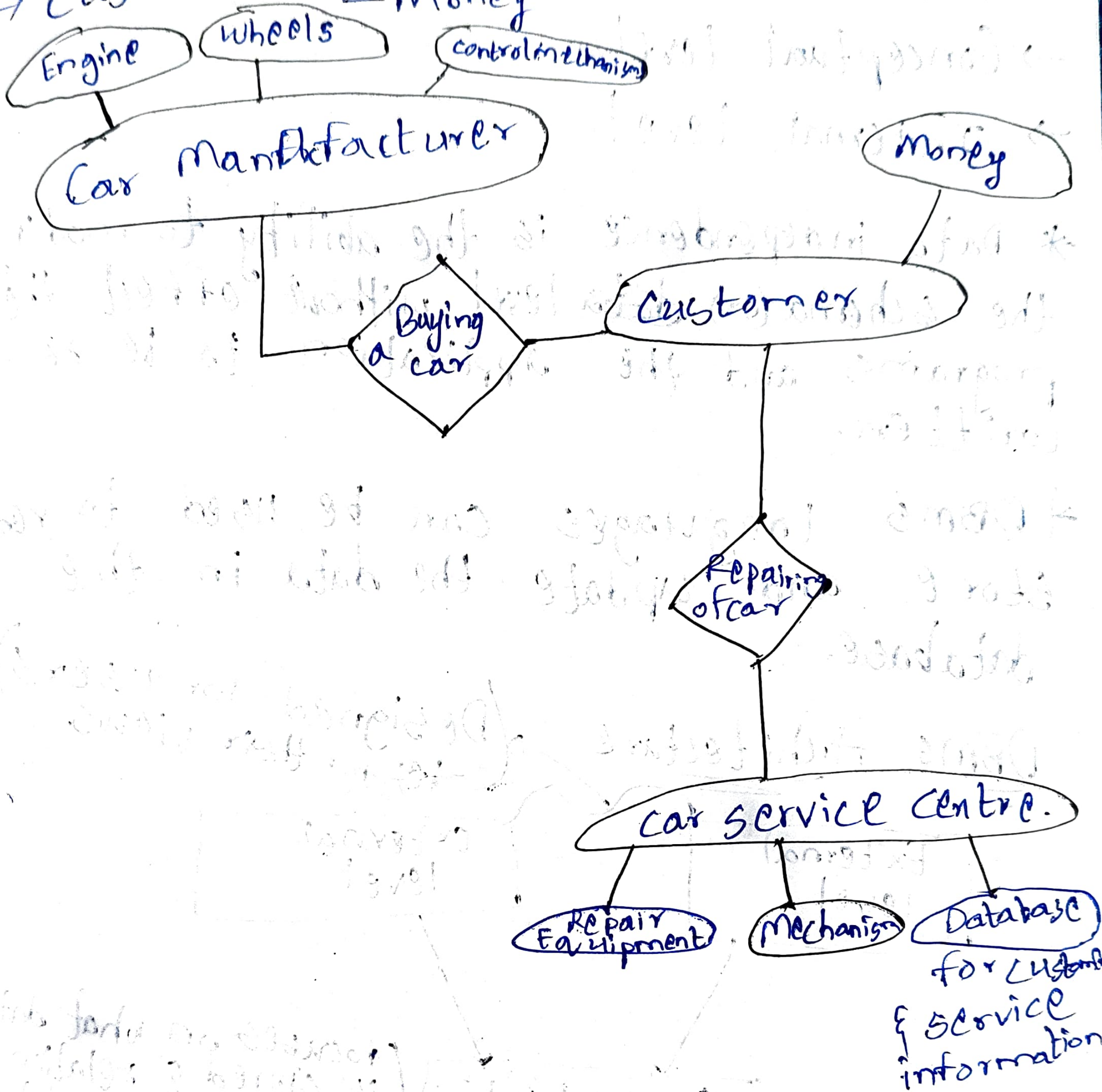



## Applications Areas of DBMS

→ Banking
→ Airline, Finance, Manufacturing, Universities
Credit & Transactions, Telecommunications

Q:) A popular car service centre wanted to maintain the complete information about the customers and serviced cars. Car manufacturer makes different model cars and sell the various customers. Customer owns one or more cars with same or different companies. Customer approaches XYZ car service centre for repair. Service centre assign a mechanism for the repairs cars belongs to customer. After the service customers will be changed.

→ Car service centre — Payment, Repair Equipment
→ Car manufacturer — Engine, wheels, interior, & all other other control mechanisms
→ Customers — car (for repair), Money



# Three Level Architecture

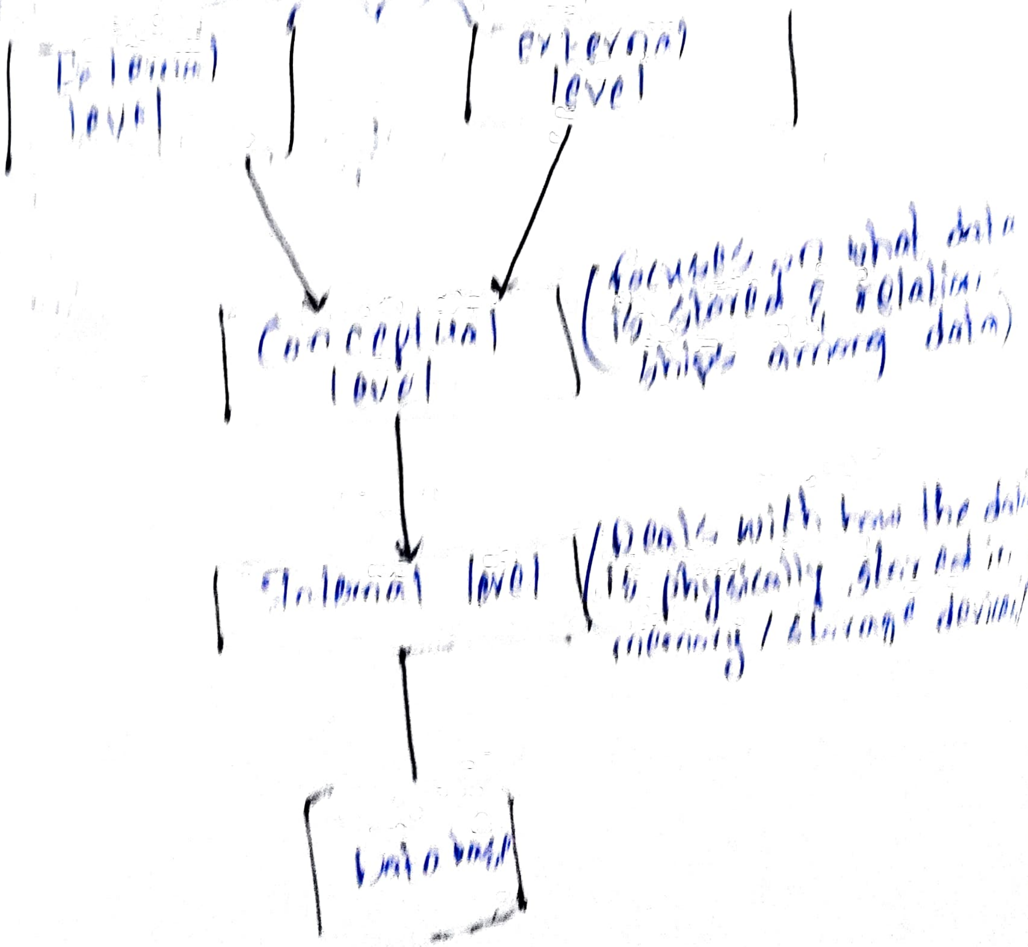
* DBMS provides three level of data
  - External level
  - Conceptual level
  - Internal level

* Data independence is the ability to modify the schema (at) ~~data~~ level without affect the programs and the application to be rewritten.

* DBMS languages can be used to read, store and update the data in the database.

## DBMS Architecture

(Designed for users i.e. for their views)



External level (External Schema)

This is the highest level of database extraction.

Ex:- Each student has different view of the time table. The view of a student of B.Tech (CSE) is different from the view of the student of B.Tech (ECE).

* This level is concerned of different categories of users.

Conceptual Level (Conceptual Schema)

* All the entities & the relationships among them are included. One conceptual view represents the entire database.
* Also known as logical level.

Internal Level (Internal Schema)

* This indicates ~~#~~ how the data will be stored & describes the data structures & access methods to be used by the database.

→ Data can be stored in B-Trees & hashing.

Note:- The main purpose of the three levels of data abstraction is to achieve data independence.

## Levels of Data Independence

1) Physical Data Independence

* It is the ability to modify the physical schema without making it necessary to rewrite application programs.

2) Logical Data Independence

* This modifies the conceptual schema without rewrite application programs.

## Types of DBMS

→ Object based databases

Uses concepts such as entities, attributes & relationships.

→ Hierarchical Databases

* This is oldest database models.
* (IMS) : Information Management System is based on this model.

→ Network Databases

* Data is represented with graph.
* many to many relations (N : N)

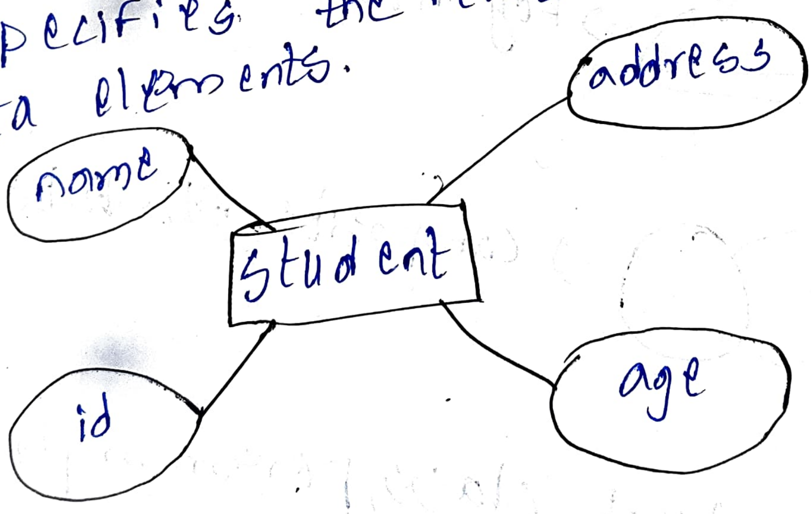
# Relational Databases

→ These store data in the form of tables.

* The DBMS architecture shows how data in the database is viewed by the users.
* It helps in implementation, design, & maintanence of a database to store & organize information for companies.

## ER MODEL IN DBMS

* ER ⇒ Entity Relationship model.
* This is a high level data model.
* specifies the relationship b/wn the data elements.




## Components of ER Diagram

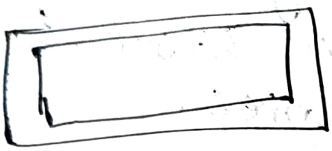
ER model

- Entity
  - Weak Entity
- Attribute
  - Key Attribute
  - Composite Attribute
  - Multivalued
  - Derived
- Relation
  - One to one
  - One to many
  - Many to one
  - Many to many

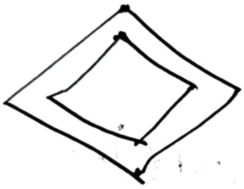
⇒ Entity.

⇒ Relationship.

⇒ Attribute.

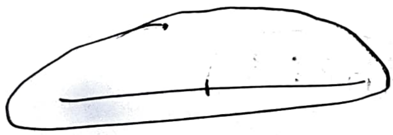


⇒ Weak entity.

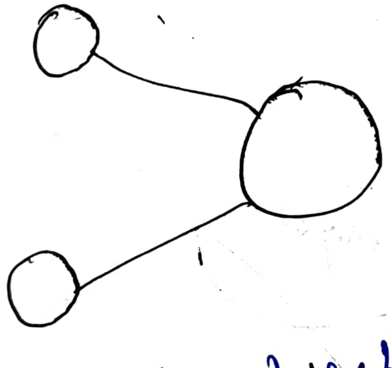


⇒ Weak entity Relationship.

⇒ Multivalue Attribute.



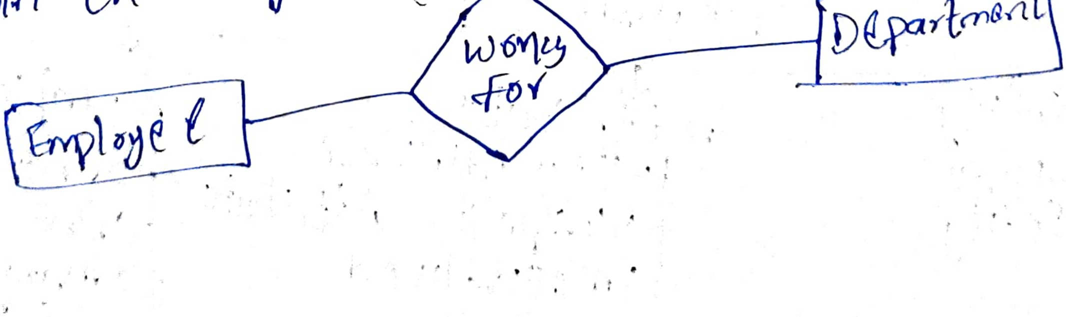
⇒ Key Attribute.



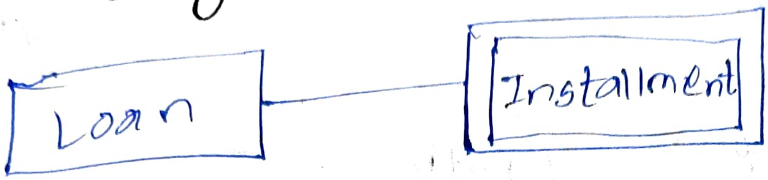
⇒ Composite Attribute.

Entity:- An object, class, person or place is an entity.

Ex:-



## Weak Entity:-



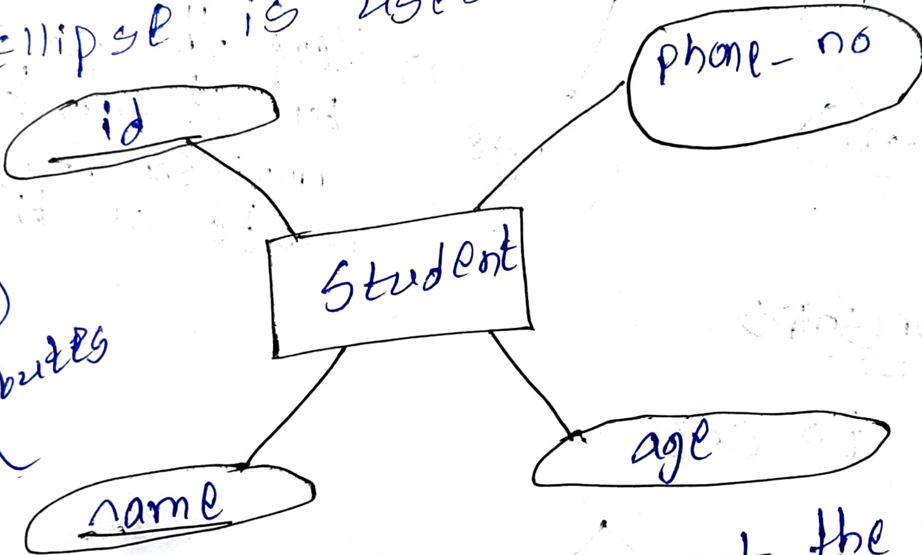

## Attribute:-

Describes the properties of an entity.

* Ellipse is used to represent attributes

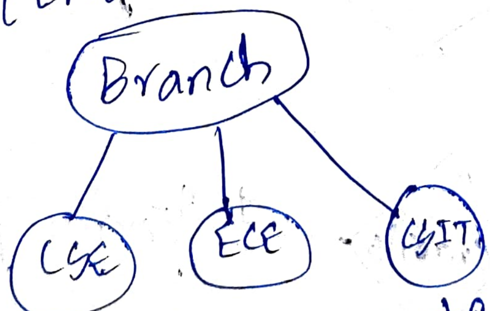
Ex:-

key attributes



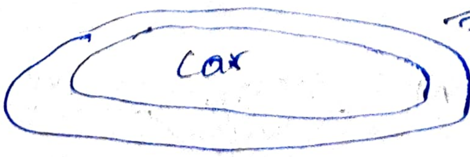

* key attributes represent the main characterstics of an entity.
* represents a key.
* Composite attributes consists of many other attributes connected.

Ex:-




* A-multivalued attribute have more than one value.

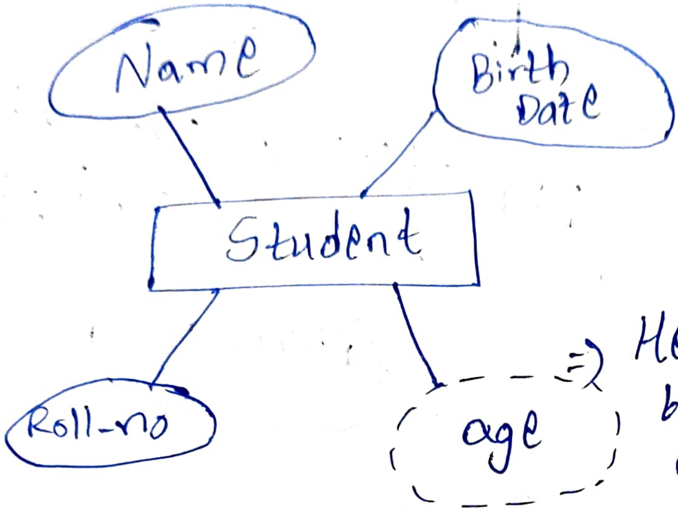
Ex:-




⇒ A customer can have more than one car.

* Derived attribute =>  => Dashed ellipse

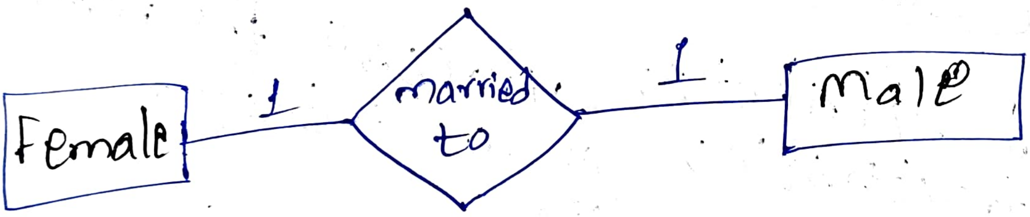
Ex:-



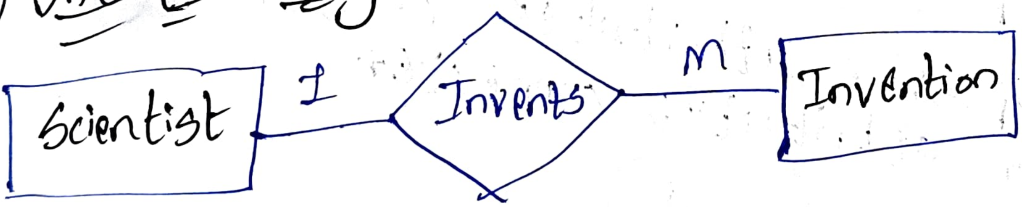

=> Here the age changes but it can be derived from date of birth.

Relationships

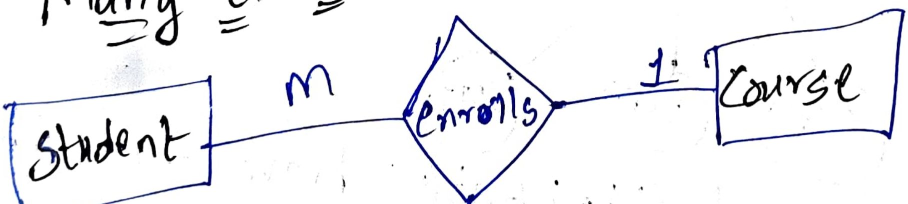
1.) One to one



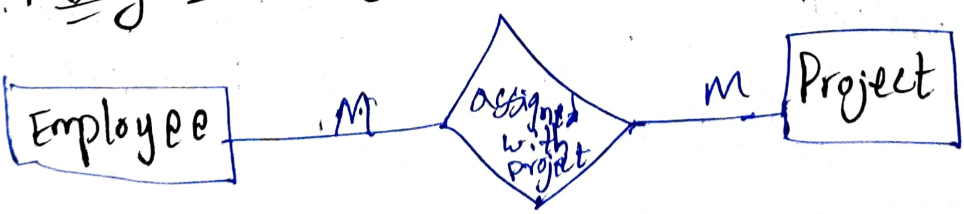

2.) One to many




3.) Many to one




4.) Many to Many



# Constraints

* These define rules that the database must follow.

## 1) Entity Integrity Constraints

* These are rules in relational database.
* Every table must have a primary key, & that key cannot be NULL.

## 2) Referential Integrity Constraints

* Here the foreign key in one table refers to a primary key to another table (or sometimes in the same table).

## 3) Domain Constraints

* These specify the allowable values for a column or attribute. specify columns can have only

Ex:- Domain constraint

0 to 100.

## 4) Check constraints

* These specify that a condition must be true

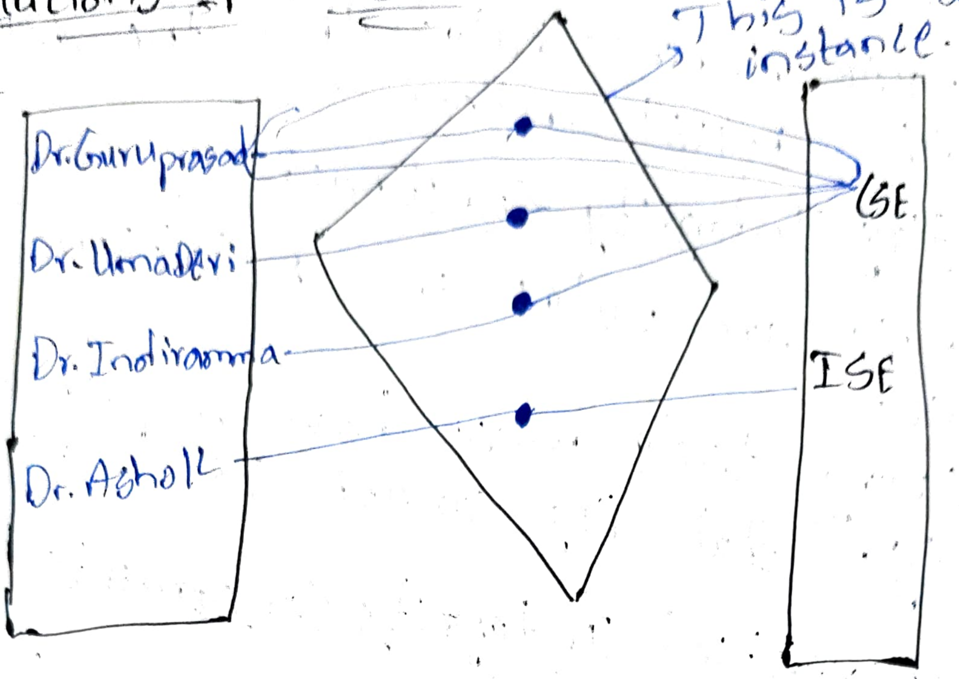
Ex:- The value of a column must be greater than zero.

## 5) Business Rules constraints

These constraints are often unique to a particular business & may not be directly related to the database structure.

## Relationship Instance

Ex:-




## Relationship Set

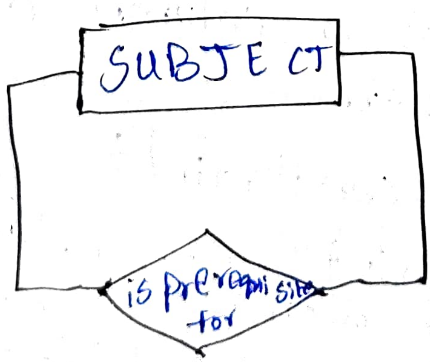
* It is a collection of relationships all belonging to one relationship type.

Ex:- In the above diagram 3 students enrolled in CSE, so 3 relationships are there in that set.

## Relationship Degree

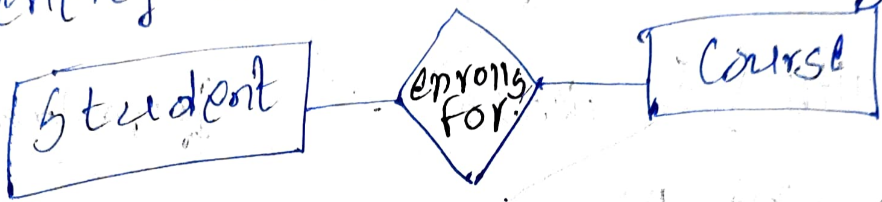
* This tells how many entities are connected together in a single relationship instance.

## Unary Relationship (Degree One)

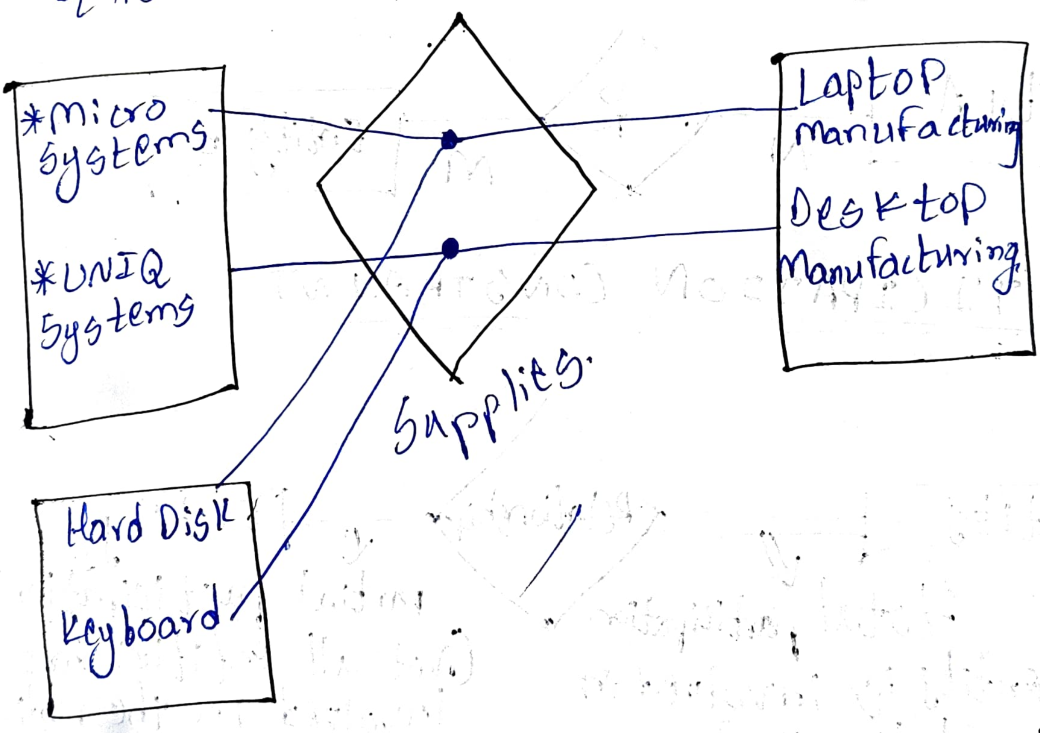



* Also known as recursive relationship.

Binary Relationship (Degree two)
An entity related to other entity.

Ex:- 


Ternary Relationship (Degree Three)
Three entities will participate in this relationship.



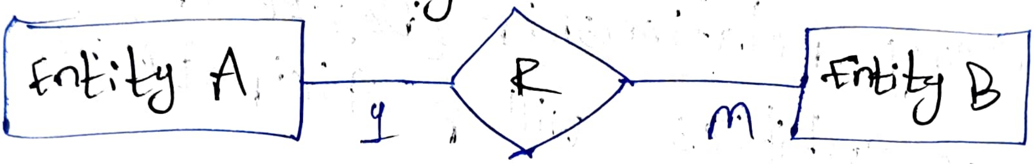

Two Types of relationship Constraints

1) Cardinality Ratios.
→ 1:1 (one to one)
→ 1:m (one to many)
→ N:m (many to many)

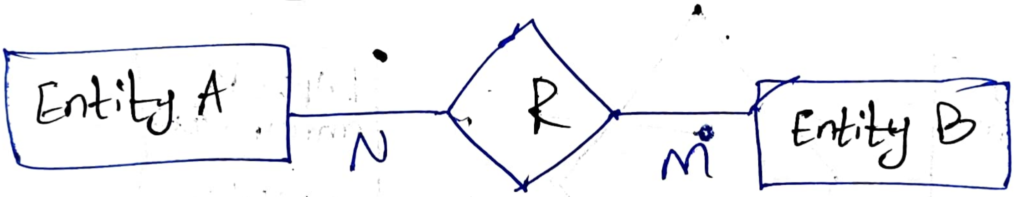
2) Participation Constraints
→ Total
→ Partial.

# CARDINALITY (one-to-one)

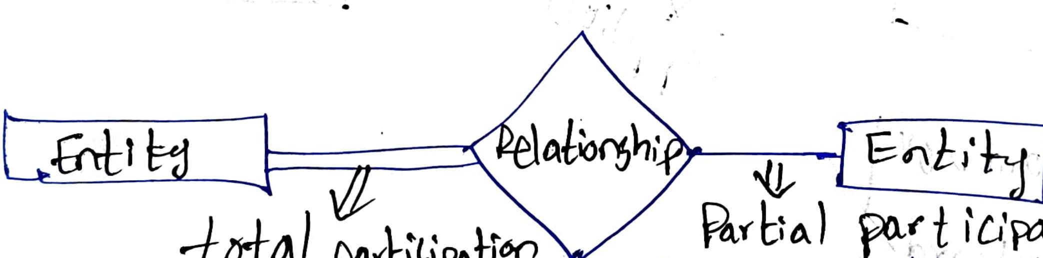

~~Many~~ One-Many




Many-Many




# PARTICIPATION CONSTRAINT



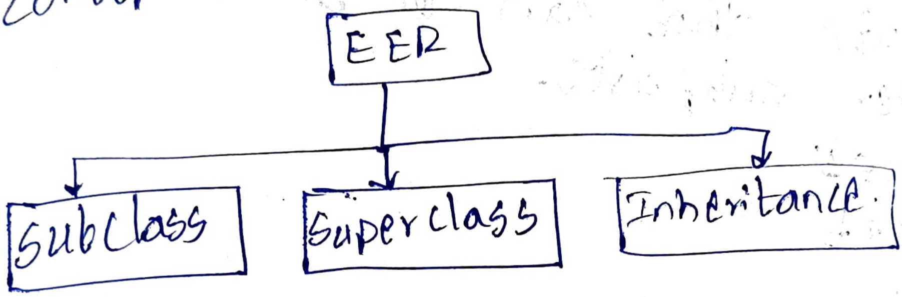

total participation
(Each entity is involved in the relationship)

Partial participation
(Not all entities are involved in the relationship).

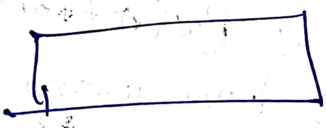
* Constraints & relationships in an ER model work together to ensure that the data in a database is accurate, consistent & effective.

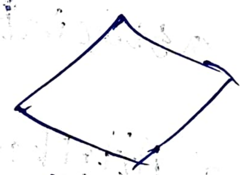
# The Extended Entity Relationship Model)

* This EER model is a conceptual data model, describes the data requirements for a new information system
* Data requirements for a database are conceptual Schema.

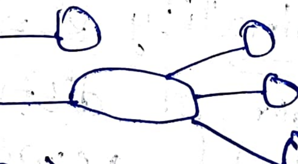



## Constructs of EER Model

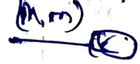
→ Entity ⇒ 

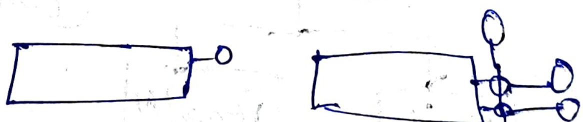
→ Relationship ⇒ 

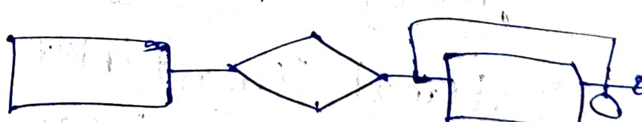
→ Simple attribute ⇒ 

→ Composite attribute ⇒ 

→ Cardinality of a relation ⇒ 


→ Cardinality of an attribute ⇒ 


→ Internal identifier ⇒ 

→ External identifier ⇒ 

→ Generalization ⇒ 

→ Subset ⇒ 

* Generalization:-

→ Abstracts the common features from multiple entities.

Ex:- Car, bike, Truck → Vehicle → Generalization

* Composite attribute:-

→ Attribute that can be divided into smaller subparts.

Ex:- Address
 → Street
 → City
 → Door No

* Cardinality of a Relationship

* This specifies how many instances of one entity can associate with another entities.

* Cardinality of a Attribute

* Number of distinct values an attribute can hold across all entities.

* Ex:- Gender → Attribute (cardinality=2).

Gender
 → Male
 → Female

Internal Identifier

* Internal identifier is an attribute that belongs entirely to the entity itself.

Ex:- Employee identified by Employee ID

* Uniqueness comes from inside the entity.

## External Identifier

This is an attribute that identifies relationship with an entity by using a another entity.

* uniqueness comes from another relation.

## Subset

→ It is a sub entity set which is derived from higher-level entity set.

Ex:-

Employee ⇒ Super class
- Manager
- Engineer
- Intern.

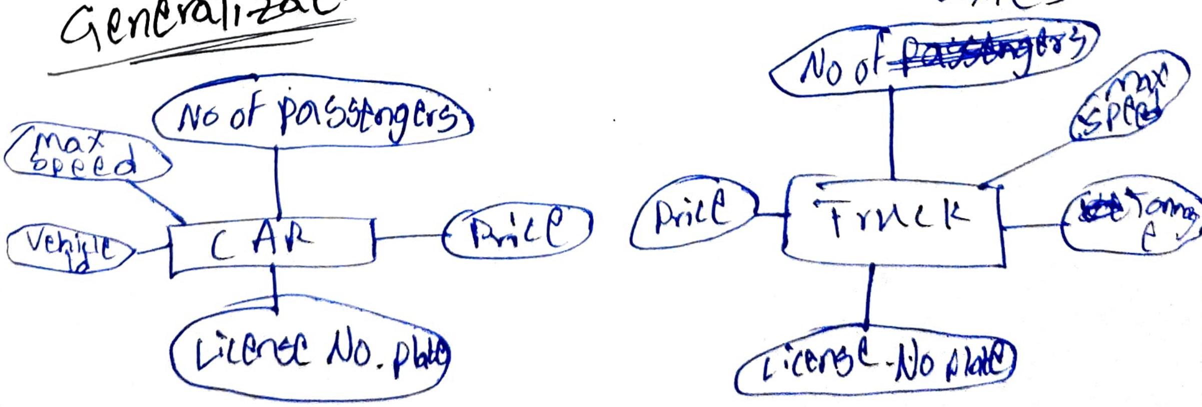
} Subclasses.

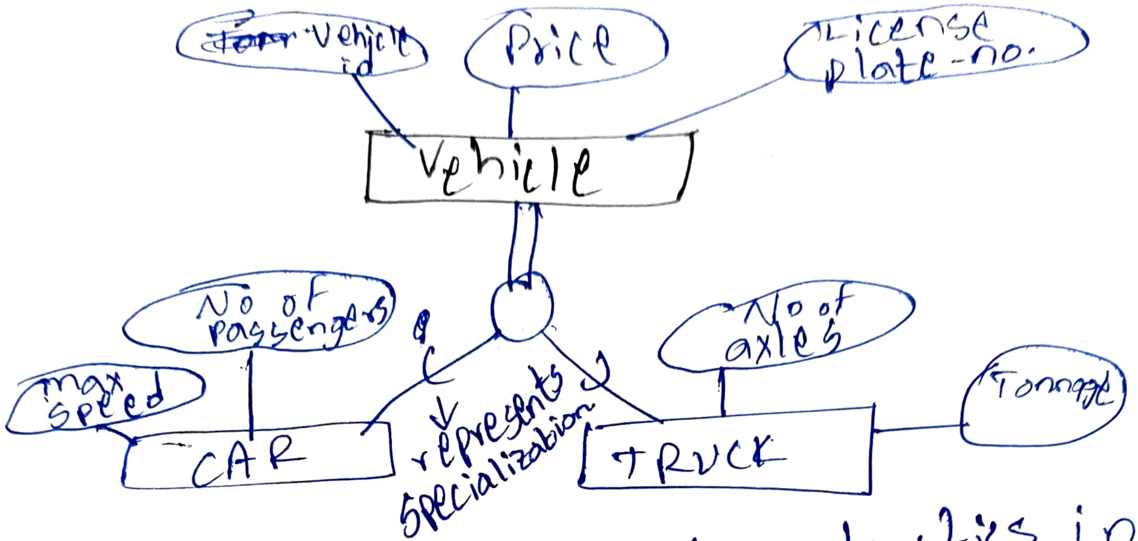
## Attribute Inheritance

* All attributes of the entity acts as a member of the superclass.
* All relationships of the entity as a member of the superclass.

* Specialization is the process of defining a set of subclasses of a superclass.

## Generalization






* Here the common charalterstics in car & truck are taken into one entity that is generalized into a vehicle which consists of car & truck entities connected which have their own set of attributes.

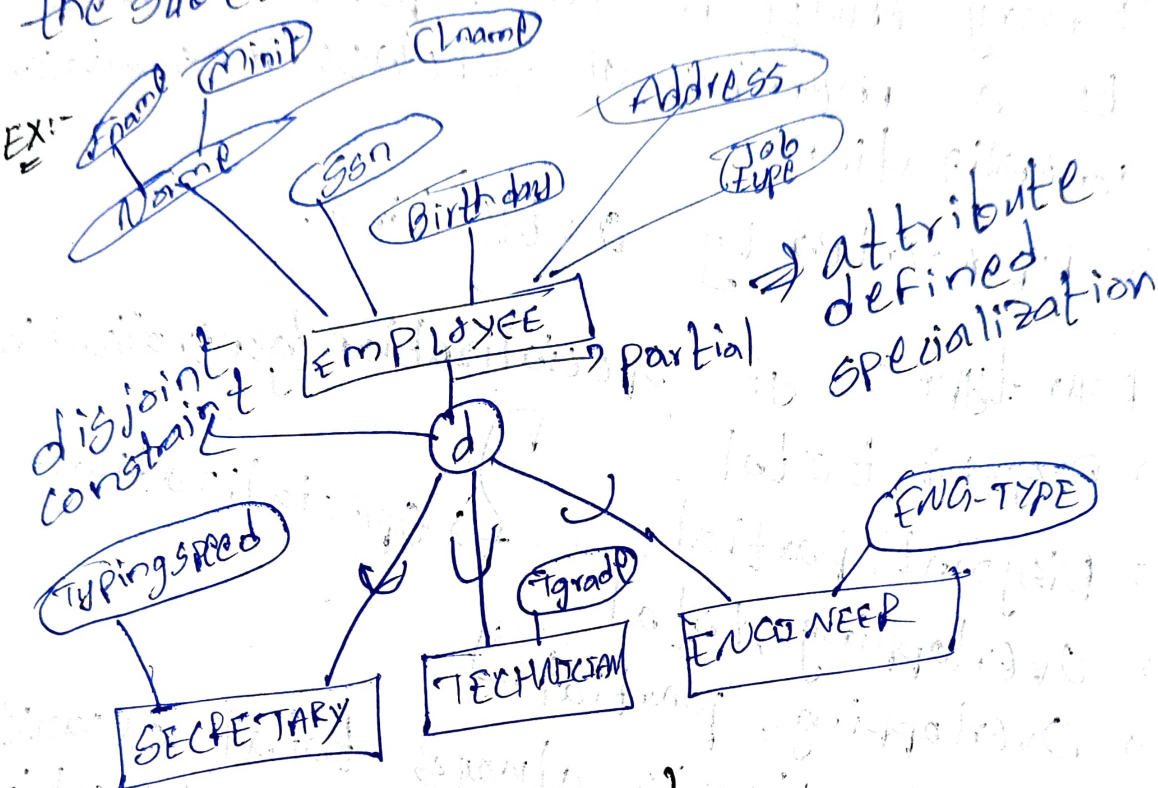
Constraints (Disjointness & Completeness).

* If we can determine that a entity can become member of each subclass they are called predicate-Condition defined subclasses.
constraint.

* If all the ~~condition~~ subclasses are specialized under the same condition of a same attributes of the superclass then they are called attribute-defined specialization.

* If no condition determines membership the subclass is user defined.

Ex:-




Disjointness Constraint

* An entity can be a member of atmost one of the subclasses of the specialization.

* If not disjoint, specialization is overlapping.

* ⇒ In this same entity can be a member of more than one subclass of the specialization.

→ 'O' in EER diagram.

# Completeness Constraint

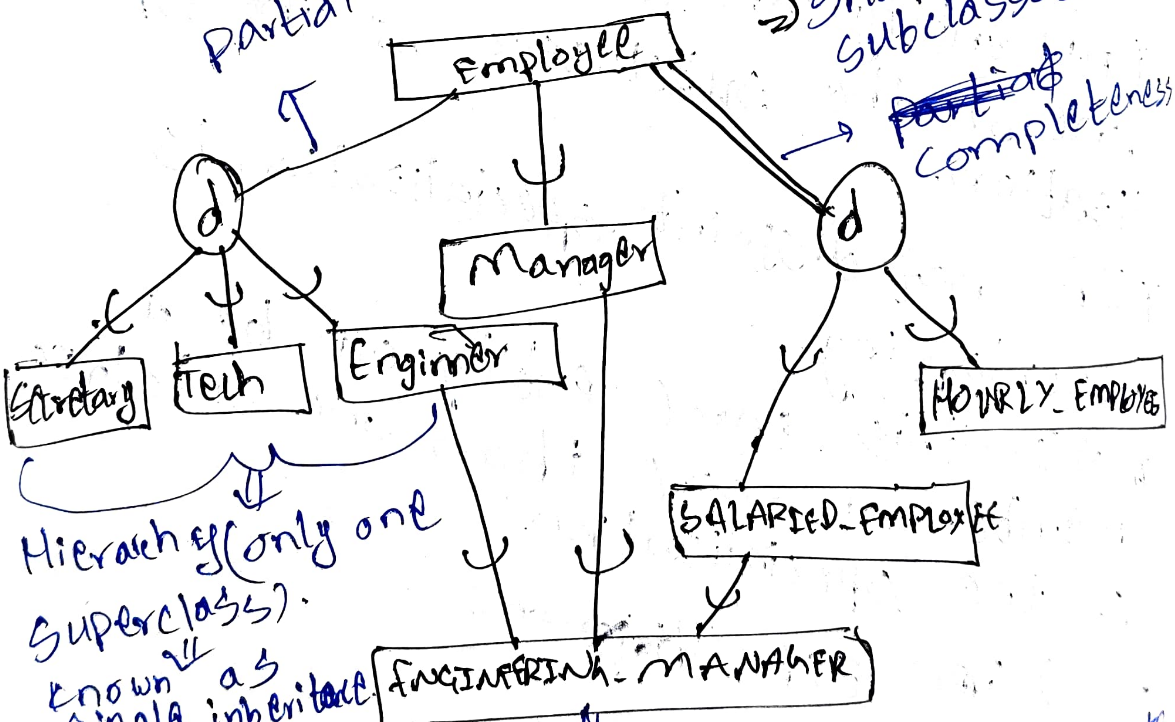
* Every entity in the superclass must be a member of subclass in specialization/generalization.
* Represented by double line.

Four types of specialization/generalization

→ Disjoint, total
→ Disjoint, partial
→ Overlapping, total
→ Overlapping, partial.

} → specialization.

* Generalization is always total because the superclass is derived from the subclasses.




Hierarchy (only one superclass). ⇓ known as single inheritance

⇓ Lattice.

known as multiple inheritance ⇐ (A sub class can be subclass for more than one superclass).

* Superclasses represent entity types.
* Subclasses are category or UNION TYPE.

* Structured Query Language (SQL) is the database Language.

Types of Languages are widely used in SQL queries:

→ DDL (Data Definition Language).
→ DML (Data Manipulation Language).
→ TCL (Transaction Control Language).
→ DCL (Data Control Language).

## DDL

This consists of the SQL commands that can be used to define the database structure or schema.

**List of DDL commands**

→ CREATE
→ ALTER : Alters the structure of the database.
→ DROP : Deletes objects from the database.
→ TRUNCATE : Removes all records from a table.
→ RENAME